

LAB4 - La gestion des processus



Table des matières

Objectifs.....	2
Rappels et mise en place de l'environnement.....	2
Gestion des processus.....	2
1Shell.....	2
2Création de processus en C.....	3
3Schéma « gestion des processus ».....	4



Objectifs

Ce « LAB » présente les notions de base de gestions de processus. Il est basé sur la mise en parallèle de notions relatives aux commandes Shell (et Scripts Shell) avec leur équivalent en langage C.

- Documents : Support Linux – Les processus.
- Notions abordées : le « scheduler », les signaux, la gestion synchrone et asynchrone, /proc
- Commandes et fonctions exploités : ps, pstree, top, trap, kill, fork(), exec().
- **Travail à rendre :** Vous devrez répondre directement à plusieurs questions au sein de ce document, vous le déposerez sur Moodle sous le nom : **LAB4_NOM.pdf**. Il y a un code C à écrire dans la question 2, vous le déposerez sur Moodle sous le nom **part-2_NOM.c**.

Gestion des processus

1 Shell

Lisez le document « Support Linux – Les processus »

Répondez aux questions suivantes dans les zones (____) en utilisant **une couleur distincte**.

- Dans un terminal : lancez un programme (*firefox*, *gedit*, etc.). En se lançant, votre éditeur affiche (ou pas...) des informations sur le canal des erreurs. Comment feriez-vous pour qu'il n'affiche plus ces erreurs ? (**commande : audacity 2>/dev/null**). Testez. Info : il est toujours intéressant de lancer les applications graphiques à partir d'un terminal, car cela permet de voir des dysfonctionnements souvent cachés par les interfaces graphiques (idem sous Windows).
- Pourquoi n'avez-vous plus la main sur votre terminal ? (**il y a déjà un processus en cours qui est exécuté en premier plan**). Lancez en aveugle la commande *id*. Fermez votre éditeur via ses menus graphiques. Cela est-il normal ? (**oui**). Relancez-le de manière asynchrone et sans affichage des messages d'erreur (**commande : audacity > /dev/null 2>&1 &**).
- Définissez les 13 champs affichés par la commande *ps -faxl* (**F : Processus, UID : Propriétaire, PID : Identifiant, PPID : Identifiant parent, PRI : Priorité, NI : Priorité initial, valeur de nice, VSZ : Espace de memoire virtuel, RSS : Espace de mémoire physique, WCHAN : événement attendu par le processus, STAT : Etat actuel du processus, TTY : Terminal d'attachement, TIME : Temps d'exécution cumulé de la commande, COMMAND : Commande avec le chemin,**). Lancez un éditeur (*gedit*, *kate*, etc.), quels sont le PID et le PPID de votre éditeur ? (**PID :8164, PPID : 6982**). Quel est le PPID du père de votre éditeur ? (**6959**)
- Affichez la liste des signaux disponibles sur votre système (**kill -l**). En tant que père de votre éditeur, votre shell peut lui envoyer des signaux. Fermer votre éditeur en lui envoyant le signal d'interruption SIGINT (**commande : kill -2 8164**).
- Le PPID d'un shell correspond au processus qui gère le terminal et les entrées clavier. Quel est-il sur votre système ? (**6959**). Quelle commande permet de connaître les caractéristiques du terminal : (**stty -a**). En analysant cette commande, vous pourrez connaître les séquences de touches associées à certains envois de signaux. Quelle séquence pour le signal SIGINT ? (**^C**). Testez.
- Lancez votre éditeur de manière synchrone (vous n'avez donc plus la main). Lancez-lui le signal de suspension (SIGTSTP) de deux manières différentes (**1 : kill -20 53061**) (**2 : ^Z**). Vous reprenez ainsi la main. Votre éditeur est bien toujours présent, mais comme le processus ne passe plus dans la boucle du microprocesseur, il est devenu inerte (sans activité). Vérifiez. Vous pouvez le réactiver en lui envoyant le signal (SIGCONT) de deux manières différentes (**1 : kill -18 53061**) (**2 : fg**). Testez. Vous pouvez réactiver tous les processus stoppés via la commande *bg* (background). INFO : vous connaissez maintenant un bon moyen de reprendre la main sur votre terminal quand vous lancez un processus graphique.

- Qu'arrivera-t-il à votre éditeur si son père meurt ? (**il meurt aussi**). Testez en fermant le terminal. Rouvrez un terminal et lancez-le de manière asynchrone afin qu'il ne dépende plus de son père en cas de mort subite de ce dernier (**commande : `nohup gedit &`**). Vérifiez en fermant le terminal. Qui est devenu PPID de votre éditeur ? (**`/lib/systemd/systemd --user`**).
- Les signaux sont traités par les processus via les standards de programmation. Le « shell » est un processus comme un autre. Ses programmeurs ont prévu que l'utilisateur puisse intercepter les signaux afin de modifier l'action prévue normalement. La commande `trap` qui est interne au « shell » permet ainsi d'intercepter et de reprogrammer l'action des signaux. Ainsi, dans un « shell », la commande `trap 'ls' num_signal` permet d'intercepter le signal numéro `num_signal` quand il est reçu afin de lancer la commande `ls` au lieu du traitement normalement prévu. À quelle séquence de touche correspond le signal `SIGINT` ? (**`^C`**). Reprogrammer l'action liée à ce signal afin de lancer la commande `ps`. Testez. Comment pouvez-vous réinitialiser cette programmation à sa valeur par défaut ? (**`trap 2`**).
- Créez un « script shell » qui affiche la liste des usagers de votre système. Cette liste est dans le fichier « `/etc/passwd` » (vous pouvez judicieusement exploiter la commande `cut`). Dans un terminal (et donc un shell), modifiez le comportement du signal numéro 3 afin de lancer votre script. Ouvrez un deuxième terminal. Déterminez le PID du premier terminal et envoyez-lui le signal numéro 3. Faites une capture d'écran des 2 terminaux. Supprimez l'interception du signal numéro 3.

```

rootor@victor-G5-5590:~/TP/A457/systeme/TD4$ ^\
daemon
bin
sys
sync
games
man
lp
mail
news
uucp
proxy
www-data
backup
list
irc
gnats
nobody
systemd-network
systemd-resolve
systemd-timesync
messagebus
syslog
_apt
tss
uidd
tcpdump
avahi-autoipd
usbmux
rtkit
dnsmasq
cups-pk-helper
speech-dispatcher
avahi
kernoops
saned
nm-openvpn
hplip
whoopsie
colord
geoclue
pulse
gnome-initial-setup
gdm
victor
nvidia-persistenced
systemd-coredump
systemd-oom
sssd
fwupd-refresh
snapd-range-524288-root
snap_daemon

```

```

victor@victor-G5-5590:~$ kill -3 7914
victor@victor-G5-5590:~$ 

```

- Quand un processus reçoit le signal numéro 1 (SIGHUP), il se ferme normalement (HUP = Hang UP = raccrocher). À partir du deuxième terminal, lancez le signal numéro 1 sur le premier terminal. Rouvrez un nouveau terminal et interceptez le signal 1 afin qu'il ne se ferme plus lors de la réception de ce signal. Testez. Quel signal faudrait-il envoyer pour fermer un processus qui ne veut plus rien savoir ? (9). Qui traite en fait ce signal ? (kernel). INFO : C'est parfois le seul moyen de supprimer un processus planté (boucle infinie par exemple).

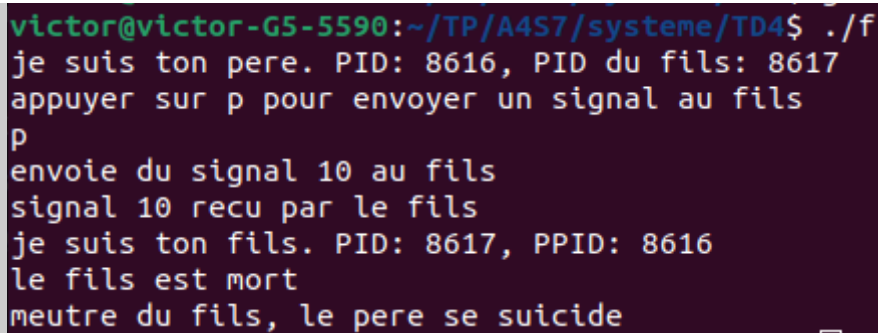
Gestion interactive : Sous KDE, GNOME : <CTRL> + <ECHAP>. Sous Windows : <CTRL> + <ALT> + <Suppr>. Sous l'environnement minimaliste LXDE : commande « lxtask ». Dans un terminal texte : commande top (puis « h » pour connaître les interactions possibles).

2 Création de processus en C

Dans les LAB précédents, vous avez exploité la fonction `system(commande)` de la librairie standard du C (stdlib.h) qui lance un « shell » à l'intérieur duquel votre commande est exécutée. Votre programme appelant attend la fin de cet appel pour poursuivre son exécution. L'enchaînement est donc synchrone. Vous allez maintenant exploiter les possibilités du système multi-processus qui permet de créer un nouveau processus géré de manière asynchrone et donc traité en parallèle (fonction `fork()` et `exec()`).

Réalisez un programme en C qui crée un processus fils. Le père affiche son PID et le PID de son fils et attend que vous tapiez la touche « p ». Le processus fils affiche son PID et son PPID. Il ne fait qu'attendre la réception d'un signal de son père (que vous définirez) pour mourir. Le père envoie ce signal quand vous tapez la touche « p ». Quand il s'est assuré que son fils est bien mort, il quitte aussi cette dure vie.

Réalisez une capture d'écran de son exécution.



```
victor@victor-G5-5590:~/TP/A4S7/systeme/TD4$ ./f
je suis ton pere. PID: 8616, PID du fils: 8617
appuyer sur p pour envoyer un signal au fils
p
envoie du signal 10 au fils
signal 10 reçu par le fils
je suis ton fils. PID: 8617, PPID: 8616
le fils est mort
meutre du fils, le pere se suicide
```

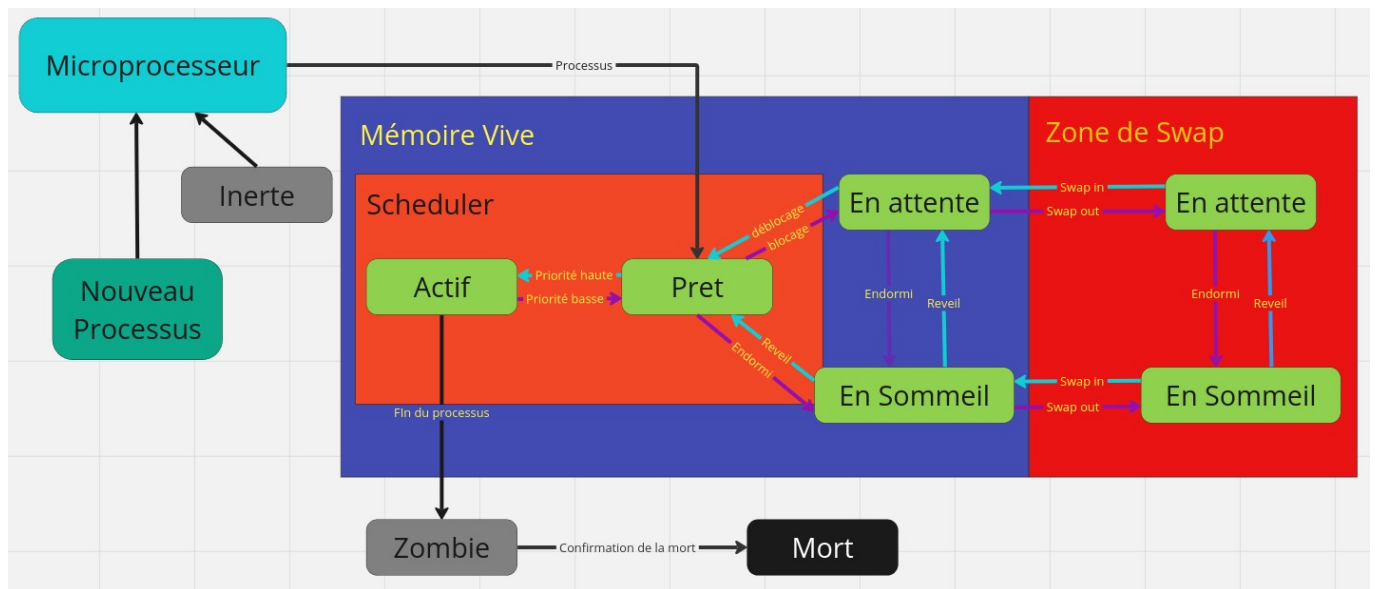
Une fois le fils créé, le père et le fils veulent écrire « je suis le (père/fils) et mon PID est ... » dans un même fichier. Ils ne peuvent cependant pas le faire en même temps...

BONUS : Ajoutez une méthode de résolution de ce problème d'accès **concurrent** à une ressource.

Rendez ce code C sur Moodle dans un fichier part-2_NOM.c.

3 Schéma « gestion des processus »

Réalisez **ci-dessous** le **schéma** de gestion des processus sous Linux. Vous tenterez de faire apparaître les termes suivants : « microprocesseur », « mémoire vive », « zone de swap », « scheduler », « inerte », « en attente », « en sommeil », « actif », « mort », « zombie », « priorité ». Dans un court **texte**, vous expliquerez la vie trépidante d'un processus.



Un processus débute sa vie lorsque le microprocesseur l'envoie s'exécuter dans la mémoire vive. Il rentre tout d'abord dans l'état prêt. Si un processus est dans cet état il va être ordonnancer avec tout les autres processus prêt en fonction de sa priorité et de son arrivé. Une fois que c'est son tour, il aura du temps actif, c'est à dire du temps pour exécuter son processus. Si il n'a pas fini dans le temps qui lui a été imparti, il retourne dans l'état prêt. Si au cours de son exécution il subit un blocage, par exemple d'E/S, il sera mis dans l'état attente. Si jamais le processus n'est pas utilisé pendant un certain temps, il sera mis à l'état en sommeil. Si la mémoire vive commence à saturé le processus pourra être mis en zone de swap, une mémoire virtuelle permettant de libérer la mémoire vive. Une fois qu'un processus a fini son travail, il sera mis à l'état zombie où il attendra la confirmation de sa fin de processus et sera mis à mort.